

ARYABHATTA KNOWLEDGE UNIVERSITY, PATNA

BCA — SEMESTER III

SOFTWARE ENGINEERING

EXAM NOTES — BLACK & WHITE EDITION

Paper Code: **303304** | Full Marks: **60** | Time: **3 Hours**

PYQ Years: **2014 • 2017 • 2018 • 2019 • 2020 • 2022 • 2024 • 2025**

CONTENTS

Unit	Topic	MCQs	4-Mark Qs	12-Mark Qs
Unit 1	Introduction to SE + Life Cycle Models	10	5	3
Unit 2	Software Project Management + SRS	10	5	3
Unit 3	Software Design	10	5	3
Unit 4	Coding, Testing + Software Reliability	10	5	3
Unit 5	Software Quality Assurance + Maintenance	10	5	3
Most Repeated	15 Topic Frequency Table	-	-	-
Cheat Sheet	6 Quick-Reference Boxes	-	-	-

TOP 6 VERY VERY IMPORTANT (★VVI) TOPICS:

- Waterfall Model (7x repeated) — Know all 6 phases, advantages, disadvantages
- Black Box vs White Box Testing (6x) — EP, BVA, Statement/Branch/Path coverage
- SRS Document (6x) — IEEE 830 structure: Introduction + Overall Description + Specific Requirements
- COCOMO Model (5x) — Formula: $\text{Effort} = a \times (\text{KLOC})^b$, three categories
- DFD Levels (5x) — Context Diagram (L0), Level 1, Level 2, balancing, 4 symbols
- Spiral Model (5x) — 4 quadrants, risk analysis mechanism, radial+angular dimensions

UNIT 1: Introduction to SE + Life Cycle Models

Characteristics · Emergence · Software Metrics · Waterfall · Prototype · Spiral Models

SECTION A — MCQ (2 Marks Each) | Unit 1

1. What is the main concern of the software engineering area? [PYQ 2014]

- (i) Hardware configuration
- ✓ (ii) Software Production ← Correct
- (iii) Network Configuration
- (iv) Software Reusability

Explanation: SE focuses on producing reliable, efficient SOFTWARE — not hardware or networks.

2. SDLC stands for? [PYQ 2018]

- (i) System Development Life Cycle
- ✓ (ii) Software Development Life Cycle ← Correct
- (iii) Software Design Life Cycle
- (iv) System Design Life Cycle

Explanation: SDLC = Software Development Life Cycle — framework of phases from planning to maintenance.

3. The Spiral model has two dimensions, namely ____ and _____. [PYQ 2018]

- (i) Diagonal and Angular
- (ii) Radial and Perpendicular
- ✓ (iii) Radial and Angular ← Correct
- (iv) Diagonal and Perpendicular

Explanation: Radial axis = cumulative cost; Angular axis = progress through the iteration.

4. Requirements can be refined using which model? [PYQ 2014]

- (i) Waterfall Model
- ✓ (ii) Prototyping Model ← Correct
- (iii) Evolutionary Model
- (iv) Spiral Model

Explanation: Prototype model refines unclear requirements by letting users test a working model.

5. Waterfall model is NOT suitable for which type of project? [PYQ 2019]

- (i) Small stable projects
- (ii) Well-defined projects
- ✓ (iii) Projects needing accommodating changes ← Correct
- (iv) Short-duration projects

Explanation: Waterfall is rigid — cannot accommodate changing requirements without high cost.

6. The most important feature of the Spiral model is? [PYQ 2019]

- ✓ (i) Risk Management ← Correct
- (ii) Efficiency Management
- (iii) Quality Management

(iv) Performance Management

Explanation: Spiral model explicitly performs RISK ANALYSIS + MITIGATION at every iteration.

7. Which is the FIRST step in the Software Development Life Cycle? [PYQ 2018]

(i) Analysis

(ii) Design

✓ (iii) Problem/Opportunity Identification ← Correct

(iv) Development & Documentation

Explanation: SDLC starts with identifying the problem/opportunity before any analysis or coding.

8. Which software life cycle model is best for large, complex, high-risk projects?

(i) Waterfall Model

(ii) Prototype Model

✓ (iii) Spiral Model ← Correct

(iv) Big Bang Model

Explanation: Spiral handles risk at every iteration — ideal for large, complex, high-risk projects.

9. The Prototype model is best used when?

(i) Requirements are perfectly clear

✓ (ii) Requirements are unclear or incomplete ← Correct

(iii) Project size is very large

(iv) Team has no experience

Explanation: Prototyping is best when requirements are fuzzy — users test the prototype to clarify needs.

10. Software differs from hardware because software:

(i) Is manufactured in factories

(ii) Physically wears out with use

✓ (iii) Does not deteriorate physically ← Correct

(iv) Cannot be replicated

Explanation: Hardware wears out physically; software does NOT wear out but may deteriorate due to modifications.

SECTION B — 4-Mark Questions | Unit 1

Q1. What is a Prototype? Under what circumstances is it beneficial to construct a Prototype? [PYQ 2014]

Introduction: A Prototype is a preliminary working model of software built to validate requirements and gather user feedback before full development.

Key Points:

- **When Requirements are Unclear:** Users cannot clearly define needs; prototype helps them visualize and articulate requirements.
- **Complex UI Systems:** For applications where look-and-feel is critical — prototypes enable early design feedback.
- **High-Risk Projects:** Validates technical feasibility before committing full resources.
- **Innovative Systems:** New concepts with no precedent benefit from early validation via prototyping.
- **Multiple Stakeholders:** Provides a common reference point when stakeholders have conflicting ideas.

Conclusion: Prototyping reduces requirement mismatches, catches design flaws early, and significantly lowers costly rework in later development stages.

Q2. What is a Process Model? Explain the Waterfall Model and its limitations. [PYQ 2017, 2020]

Introduction: A Process Model is a structured framework defining how software development activities are organized and sequenced.

Key Points:

- **Waterfall Phases:** Requirements Gathering → System Design → Implementation → Testing → Deployment → Maintenance
- **Limitation 1 — Rigid:** Once a phase is complete, going back is difficult and expensive.
- **Limitation 2 — Late Defect Detection:** Bugs found only in testing phase when fixing is costly.
- **Limitation 3 — No Early Product:** Working software not visible until very late in the project.
- **Limitation 4 — Requirements assumed stable:** Fails badly when requirements change during development.

Conclusion: Waterfall works for small, well-understood projects with stable requirements. For dynamic, large-scale systems, more flexible models (Spiral/Agile) are preferred.

Q3. Distinguish between the Waterfall Model and the Spiral Model. [PYQ Multiple PYQs]

Introduction: Both are Software Life Cycle Models but differ fundamentally in approach, flexibility, and risk handling.

Key Points:

- **Approach:** Waterfall = Linear Sequential | Spiral = Iterative Risk-Driven
- **Risk Handling:** Waterfall = Poor (risks found late) | Spiral = Excellent (at every iteration)
- **Flexibility:** Waterfall = Rigid (no going back) | Spiral = Very flexible
- **Customer Involvement:** Waterfall = Start and end only | Spiral = Active in each iteration
- **Best For:** Waterfall = Small stable projects | Spiral = Large complex high-risk projects

Conclusion: Waterfall suits stable, well-understood requirements; Spiral suits complex projects needing continuous refinement and risk management.

Q4. What is Software Prototyping? Give its advantages and disadvantages. [PYQ 2022]

Introduction: Software Prototyping is the process of building a quick preliminary version of software to validate requirements and design before full development.

Key Points:

- **Advantage 1:** Early user feedback reduces misunderstandings and errors.
- **Advantage 2:** Clarifies ambiguous requirements through concrete interaction.
- **Advantage 3:** Reduces project risk through early feasibility validation.
- **Disadvantage 1:** Scope creep — users keep requesting more features.
- **Disadvantage 2:** Prototype code may be poorly structured; risk of converting it to final product.

Conclusion: Prototyping is powerful when managed properly but can lead to scope creep and budget overruns if the process is not controlled with clear objectives and timelines.

Q5. Describe the characteristics of software that differentiate it from hardware. [PYQ 2019, 2020]

Introduction: Software has unique properties that fundamentally distinguish it from physical hardware products and traditional engineering artifacts.

Key Points:

- **Developed, not manufactured:** Software is created through intellectual work, not assembled in factories.
- **Does not wear out physically:** Unlike hardware, software does not deteriorate with use.
- **Custom-built:** Most software is built for specific organizational needs — not off-the-shelf components.
- **Intangible:** Software cannot be seen or touched — it is logic, not physical matter.
- **Easily replicated:** Software can be duplicated at near-zero marginal cost.

Conclusion: Understanding these unique characteristics is essential for applying correct engineering principles to software development, as hardware engineering approaches often do not directly translate.

SECTION C — 12-Mark Questions | Unit 1

Q1. Explain the classical Waterfall Model with activities undertaken during each phase. [PYQ 2018, 2020, 2024]

Introduction: The Waterfall Model (Winston Royce, 1970) is the oldest and simplest software life cycle model. It follows a linear, sequential approach where each phase must be fully completed before the next begins — like water flowing downward.

Answer:

- **Phase 1 — Requirements Gathering & Analysis:** Collect ALL user requirements. Produce SRS (Software Requirements Specification). Errors here multiply downstream. Critical for project success.
- **Phase 2 — System Design:** Architects design system architecture, data structures, algorithms, UI. Produces HLD (High-Level Design) and LLD (Low-Level Design). Blueprint for developers.
- **Phase 3 — Implementation (Coding):** Developers write actual code based on design documents. Produces software modules. Most visible and longest active-development phase.
- **Phase 4 — Testing & Verification:** QA team tests each module (Unit Testing), combined modules (Integration Testing), then the full system (System Testing). Identifies and removes defects.
- **Phase 5 — Deployment:** Tested software released to end users. Involves installation, data migration, user training, configuration.
- **Phase 6 — Maintenance:** Ongoing support — bug fixes, performance optimization, enhancements. Typically the longest and most expensive phase (60-80% of total lifecycle cost).

Advantages:

- Simple, easy to understand and manage
- Well-defined phases with comprehensive documentation
- Works well for small projects with stable requirements

Disadvantages:

- Rigid — cannot easily accommodate requirement changes
- Late defect detection = very high cost to fix
- No working product until end of project; high end-stage risk

Diagram / Representation:

```
Waterfall Model:  
[Requirements] → [System Design] → [Implementation]  
↓  
[Maintenance] ← [Deployment] ← [Testing]
```

Conclusion: The Waterfall Model is best for small, well-understood projects with stable, fully-defined requirements. For large, complex systems with evolving requirements, more flexible models like Spiral or Agile are preferred.

Q2. Explain the Spiral Model. How are risks handled in spiral mode of software development? [PYQ 2014, 2017, 2022]

Introduction: The Spiral Model (Barry Boehm, 1986) is an iterative, risk-driven approach that combines the structured aspects of Waterfall with the iterative nature of Prototyping. Most suitable for large, complex, high-risk software projects.

Answer:

- **Structure:** Multiple iterations (spirals). Each loop produces an increasingly complete version while explicitly addressing risks.
- **Quadrant 1 — Planning & Objectives:** Define goals, constraints, and alternatives for the current iteration. Identify what will be delivered.
- **Quadrant 2 — Risk Analysis & Mitigation:** IDENTIFY all possible risks. Evaluate their impact. Develop strategies to MITIGATE or eliminate them. This is the HEART of the Spiral model.
- **Quadrant 3 — Development & Testing:** Build and verify the deliverable — could be a prototype or actual software. Testing is performed within this quadrant.
- **Quadrant 4 — Planning for Next Iteration:** Review with stakeholders. Plan the next iteration based on feedback and remaining work.

- **Risk Handling Mechanism:** Risks identified BEFORE each development cycle. Higher-risk items addressed in early spirals; lower-risk in later spirals. Prototyping used during risk analysis to evaluate feasibility.

Advantages:

- Best risk management of all models
- Flexible — accommodates changing requirements
- Customer sees working software early

Disadvantages:

- Complex to manage; requires expert risk analysts
- Expensive — not suitable for small projects
- Risk assessment needs expertise

Diagram / Representation:

Spiral Model — 4 Quadrants per loop:

Q1: Planning & Objectives → Q2: Risk Analysis & Mitigation
 ↑ ↓
 Q4: Review & Next Plan ← Q3: Development & Testing

Start at center → spiral outward with each iteration
 Each complete loop = one iteration of software development

Conclusion: The Spiral Model's iterative risk management makes it the ideal choice for large, complex projects where uncertainty is high, requirements evolve, and risks need to be proactively managed.

Q3. Compare and contrast the Waterfall, Prototype, and Spiral models with their advantages and disadvantages. [PYQ 2020, 2022]

Introduction: Three major software life cycle models represent different philosophies for managing software development. Choosing the right model critically affects project success.

Answer:

- **Approach:** Waterfall = Linear Sequential | Prototype = Iterative Feedback | Spiral = Iterative Risk-Driven
- **Requirement Stability:** Waterfall = Must be fixed early | Prototype = Can evolve | Spiral = Can change significantly
- **Risk Handling:** Waterfall = Poor (late detection) | Prototype = Medium (early clarity) | Spiral = Excellent (each iteration)
- **Customer Involvement:** Waterfall = Start & end only | Prototype = Continuous | Spiral = Each iteration
- **Project Size:** Waterfall = Small to medium | Prototype = Small to medium | Spiral = Large & complex
- **Time to First Release:** Waterfall = Long (end of project) | Prototype = Short (early model) | Spiral = Medium
- **Documentation:** Waterfall = Very thorough | Prototype = Limited | Spiral = Moderate-High
- **Cost Predictability:** Waterfall = High | Prototype = Low | Spiral = Medium

Diagram / Representation:

Waterfall: Req → Design → Code → Test → Deploy (linear)
 Prototype: Req → Proto → Feedback → Refine → Build (feedback loop)
 Spiral: [Plan → Risk → Build → Review] × N iterations (outward spiral)

Conclusion: No single model fits all projects. Waterfall for stable & small, Prototype for unclear requirements, Spiral for large & risky. Selection must consider project requirements, team expertise, risk profile, and budget.

UNIT 2: Software Project Management + SRS

LOC · FP · COCOMO · Delphi · Halstead · Putnam's Model · SRS Documents

SECTION A — MCQ (2 Marks Each) | Unit 2

1. COCOMO stands for? [PYQ 2014]

✓ (i) Constructive Cost Model ← Correct

(ii) Constructive Code Model

(iii) Computing Cost Method

(iv) Calculated Output Model

Explanation: COCOMO = COnstructive COst MOdel by Barry Boehm (1981). Estimates effort and schedule using KLOC.

2. In size-oriented metrics, metrics are developed based on the: [PYQ 2018]

(i) Number of functions

(ii) Number of user inputs

✓ (iii) Number of lines of code ← Correct

(iv) Amount of memory usage

Explanation: LOC (Lines of Code) = size-oriented metric. Counts executable code lines to measure software size.

3. The final outcome of requirement analysis and specification phase is? [PYQ 2018]

✓ (i) The SRS document ← Correct

(ii) Code

(iii) User manual

(iv) Design document

Explanation: SRS = BLACK-BOX specification — defines WHAT system must do, produced at end of requirements phase.

4. SRS is also known as specification of? [PYQ 2014]

(i) White-box testing

(ii) Stress testing

✓ (iii) Black-box testing ← Correct

(iv) Integrated testing

Explanation: SRS = Black-box specification because it specifies behavior (I/O) without internal implementation details.

5. Which is NOT a characteristic of a good SRS document? [PYQ 2019]

✓ (i) Ambiguous ← Correct

(ii) Traceable

(iii) Concise

(iv) Verifiable

Explanation: A good SRS must be UNAMBIGUOUS (single interpretation). Ambiguity causes errors and misunderstandings.

6. If every requirement can be checked by a cost-effective process, then SRS is? [PYQ 2019]

(i) Traceable

(ii) Modifiable

(iii) Complete

✓ (iv) Verifiable ← Correct

Explanation: Verifiable = each requirement can be tested cost-effectively to confirm it has been met by the system.

7. Which of these is a FUNCTIONAL requirement? [PYQ 2018]

(i) Maintainability

(ii) Portability

✓ (iii) User login authentication ← Correct

(iv) Reliability

Explanation: Functional = WHAT system does (login, calculate, store). Maintainability/Portability/Reliability = Non-functional.

8. COCOMO model was developed by?

✓ (i) Barry Boehm ← Correct

(ii) Roger Pressman

(iii) Ian Sommerville

(iv) Tom DeMarco

Explanation: Barry Boehm developed COCOMO in 1981 at TRW, analyzing 63 historical software projects.

9. FP stands for?

(i) Functional Program

✓ (ii) Function Point ← Correct

(iii) File Processing

(iv) Formal Protocol

Explanation: FP = Function Point. Measures software size from USER perspective — language independent.

10. The Delphi technique is primarily used for? [PYQ 2022]

(i) Bug tracking

✓ (ii) Cost estimation ← Correct

(iii) Testing methodology

(iv) Database design

Explanation: Delphi = expert-based COST ESTIMATION using anonymous rounds until consensus is reached.

SECTION B — 4-Mark Questions | Unit 2

Q1. List the important shortcomings of LOC (Lines of Code) as a software size metric. [PYQ 2014]

Introduction: LOC (Lines of Code) counts executable program lines. While widely used, it has significant shortcomings.

Key Points:

- **Language Dependent:** Same functionality in Java = 50 lines; in Python = 10 lines. Cross-language comparison meaningless.
- **Counting Ambiguity:** No universal standard — count comments? blank lines? data declarations?
- **Penalizes Good Programmers:** Efficient code in fewer lines gets a LOWER LOC count — perverse incentive.
- **Not Available Early:** LOC can only be measured AFTER coding — useless for early project estimation.
- **Poor Quality Indicator:** High LOC could mean poor design and redundant code, not quality software.

Conclusion: Despite its simplicity, LOC is a crude metric. Function Points (FP) is preferred as it is language-independent and measures user-visible functionality from a business perspective.

Q2. Briefly explain the Delphi cost estimation technique. [PYQ 2022]

Introduction: The Delphi Technique is a structured, expert-based estimation method that prevents bias from dominant personalities using anonymous rounds of estimation.

Key Points:

- **Step 1 — Select Experts:** Choose 3-10 subject matter experts for the estimation panel.
- **Step 2 — Round 1 (Blind):** Each expert independently provides cost/effort estimate — no group discussion allowed.
- **Step 3 — Anonymize & Share:** Coordinator compiles anonymous results (min, max, average) and shares with all.
- **Step 4 — Round 2 (Revise):** Experts revise estimates independently based on group summary feedback.
- **Step 5 — Repeat until Consensus:** Continue rounds until experts converge on agreement.

Conclusion: Delphi is most valuable for novel projects without historical data. It reduces groupthink and social pressure while encouraging independent expert reasoning.

Q3. Describe the characteristics of a good SRS (Software Requirements Specification) document. [PYQ 2020]

Introduction: A good SRS is the foundation of all software development. IEEE 830 defines seven key quality characteristics.

Key Points:

- **Correct:** Every requirement accurately reflects genuine user needs — no gold-plating or assumptions.
- **Unambiguous:** Each requirement has ONE clear interpretation — no vague terms like "fast" or "user-friendly".
- **Consistent:** No contradictions between requirements — all requirements must work together.
- **Complete:** ALL requirements documented — functional, non-functional, constraints, and assumptions.
- **Verifiable:** Each requirement testable by a cost-effective process with specific measurable criteria.
- **Modifiable:** Structured for easy updates without losing consistency — numbered, cross-referenced.

Conclusion: A well-written SRS can reduce development costs by 20-50% by preventing misunderstandings early. It serves as the contract between developers and clients throughout the project.

Q4. How to estimate the size of a project? Explain LOC and FP metrics. [PYQ 2018]

Introduction: Software size estimation is the FIRST step in project planning — it determines effort, cost, and schedule. Two main metrics are used: LOC and Function Points.

Key Points:

- **LOC (Lines of Code):** Counts physical executable code lines. Simple, easy to measure. Language-dependent. Available only after implementation. Used in COCOMO: $\text{Effort} = a \times (\text{KLOC})^b$.
- **Function Point (FP):** Measures software from USER perspective. Counts 5 types: External Inputs, Outputs, Interfaces, Internal Files, External Files — weighted by complexity.
- **FP Advantage:** Language-independent — can estimate BEFORE coding begins, useful for early planning.
- **LOC Advantage:** Simple to count and understand; useful for tracking developer productivity.

Conclusion: FP is preferred for early estimation at requirements stage; LOC is useful for tracking productivity once code exists. Both have complementary roles in project management.

Q5. Define software requirements and distinguish between functional and non-functional requirements. [PYQ 2024, 2025]

Introduction: Software Requirements describe what a system MUST DO (functional) and HOW WELL it must do it (non-functional). Both types are essential for a complete specification.

Key Points:

- **Functional Requirements (WHAT):** Specific behaviors, functions, actions the system must perform.
- **FR Example 1:** "System shall allow users to register using email and password."
- **FR Example 2:** "System shall generate monthly sales reports in PDF format."
- **Non-Functional Requirements (HOW WELL):** Quality attributes — performance, security, reliability, usability.
- **NFR Example 1:** Performance — "System shall respond to 95% of requests within 2 seconds."

- **NFR Example 2: Security** — "All passwords shall be encrypted using AES-256."

Conclusion: Both types are critical: Functional = what it does; Non-functional = how well it does it. Neglecting NFRs often leads to systems that work but fail in production due to performance or security issues.

SECTION C — 12-Mark Questions | Unit 2

Q1. Elaborate in detail the parts of the SRS document with example. [IEEE 830 Standard] [PYQ 2018, 2019]

Introduction: The SRS (Software Requirements Specification) is the most critical document in software development, acting as the contract between developers and clients. IEEE 830 standard provides the universally accepted structure.

Answer:

- **Section 1 — Introduction:** 1.1 Purpose (what problem this software solves, who reads the SRS), 1.2 Scope (what software will/won't do, project name and goals), 1.3 Definitions/Acronyms (glossary for shared understanding), 1.4 References, 1.5 Overview.
- **Section 2 — Overall Description:** 2.1 Product Perspective (system context, Level 0 DFD), 2.2 Product Functions (high-level list of capabilities), 2.3 User Characteristics (end user roles and technical level), 2.4 General Constraints (hardware limits, regulatory requirements), 2.5 Assumptions and Dependencies.
- **Section 3 — Specific Requirements (Most Detailed):** 3.1 Functional Requirements — each function described with Input/Processing/Output. Example: FR-01 Login: Input = email+password → Validate credentials → Output = auth token or error message.
- **3.2 Non-Functional Requirements:** Performance (response time, throughput), Security (authentication, encryption), Reliability (availability, MTTF), Usability (learning curve), Portability (platforms supported).
- **3.3 Interface Requirements:** User Interface (UI guidelines, screen layouts), Hardware Interface (sensors, devices), Software Interface (APIs, databases), Communication Interface (protocols).
- **SRS Characteristics (IEEE 830):** Correct, Unambiguous, Complete, Consistent, Verifiable, Modifiable, Traceable — a good SRS must satisfy ALL seven characteristics.

Diagram / Representation:

```
SRS Structure (IEEE 830):
1. Introduction
  1.1 Purpose 1.2 Scope 1.3 Definitions 1.4 References
2. Overall Description
  2.1 Product Perspective 2.2 Functions 2.3 User Characteristics
  2.4 Constraints 2.5 Assumptions
3. Specific Requirements
  3.1 Functional Requirements (FR-01, FR-02 ... each with I/P/O)
  3.2 Non-Functional (Performance, Security, Reliability, Usability)
  3.3 Interfaces (UI, Hardware, Software, Communication)
```

Conclusion: A comprehensive SRS prevents scope creep, resolves stakeholder disputes, and forms the basis for all test plans. Every requirement in the SRS must eventually be verified — no untestable requirements should be included.

Q2. Different categories of software projects in COCOMO model. Give examples for each. [PYQ 2014]

Introduction: COCOMO (Barry Boehm, 1981) provides mathematical formulas for estimating effort and schedule. Based on analysis of 63 real projects, it classifies projects into three categories.

Answer:

- **Formula:** Effort (person-months) = $a \times (\text{KLOC})^b$ | Schedule (months) = $c \times (\text{Effort})^d$
- **Category 1 — Organic (Simple):** $a=2.4$, $b=1.05$. Small teams, familiar technology, stable requirements. Example: Payroll system, simple inventory management, personal productivity tools.
- **Category 2 — Semidetached (Medium):** $a=3.0$, $b=1.12$. Mix of experienced and junior staff, moderate complexity. Example: Database management system, telecom billing, airline reservation.
- **Category 3 — Embedded (Complex):** $a=3.6$, $b=1.20$. Real-time systems, tight constraints, unique requirements. Example: Flight control software, medical monitoring devices, nuclear plant controls.
- **Worked Example (Semidetached, 50 KLOC):** Effort = $3.0 \times (50)^{1.12} = 3.0 \times 56.7 = 170.1$ person-months.
Schedule = $2.5 \times (170.1)^{0.38} \approx 20.4$ months.

- **Limitations:** Garbage-in garbage-out (needs accurate KLOC estimate). Less accurate for novel technologies. Static coefficients may not reflect modern Agile practices.

Diagram / Representation:

COCOMO Categories:
 Organic: $\text{Effort} = 2.4 \times (\text{KLOC})^{1.05}$ [small, familiar]
 Semidetached: $\text{Effort} = 3.0 \times (\text{KLOC})^{1.12}$ [medium, mixed]
 Embedded: $\text{Effort} = 3.6 \times (\text{KLOC})^{1.20}$ [complex, real-time]

Conclusion: COCOMO provides a quantitative basis for project planning. Choosing the right category (Organic/Semidetached/Embedded) is critical — using the wrong category can lead to significant estimation errors.

Q3. Define Software Requirements Engineering. Discuss functional vs non-functional requirements with examples. [PYQ 2024, 2025]

Introduction: Software Requirements Engineering (SRE) is the systematic process of discovering, analyzing, documenting, validating, and managing the requirements of a software system. It is the most critical phase of SDLC.

Answer:

- **SRE Activities:** (1) Elicitation — gathering needs via interviews, questionnaires, observation, JAD workshops; (2) Analysis — checking completeness, consistency, feasibility; (3) Specification — documenting in SRS; (4) Validation — reviewing with stakeholders; (5) Management — tracking changes and tracing throughout project.
- **Functional Requirements (FR) — Definition:** Describe WHAT the system must DO — specific functions, behaviors, data processing capabilities.
- **FR Examples:** "System shall allow users to register using email and password" | "System shall generate monthly sales report in PDF" | "ATM shall dispense cash in multiples of Rs.100" | "System shall send email notifications for order confirmation".
- **Non-Functional Requirements (NFR) — Definition:** Describe HOW WELL the system performs — quality attributes and operational constraints.
- **NFR Examples — Performance:** "System shall respond to 95% of queries within 2 seconds under 1000 concurrent users."
- **NFR Examples — Security:** "All sensitive data shall be encrypted using AES-256. Sessions shall expire after 30 minutes of inactivity."
- **NFR Examples — Reliability:** "System shall maintain 99.9% availability (max 8.7 hours downtime/year)."
- **NFR Examples — Usability:** "New users shall complete core tasks within 5 minutes without external training."
- **NFR Examples — Portability:** "Application shall run on Windows 10+, macOS 12+, and Ubuntu 22+ without modification."

Diagram / Representation:

Requirements Classification:
 Functional Requirements Non-Functional Requirements
 (WHAT it does) (HOW WELL it does it)
 - Login system - Performance
 - Generate report - Security
 - Process payment - Reliability
 - Send notifications - Usability / Portability
 Test: Can you write a pass/fail test case for it?
 Yes → Functional | No/Difficult → Non-Functional

Conclusion: Both types are essential for project success. Functional requirements drive test CASES; non-functional requirements drive ARCHITECTURE decisions. Neglecting NFRs is one of the most common and costly mistakes in software development.

UNIT 3: Software Design

Classification · Approaches · Function-Oriented Design · DFDs · Structured Design · OOD

SECTION A — MCQ (2 Marks Each) | Unit 3

1. The worst type of coupling is? [PYQ 2014,2019]

- (i) Data coupling
- (ii) Control coupling
- (iii) Stamp coupling

✓ (iv) Content coupling ← Correct

Explanation: Content coupling = WORST. Module directly modifies internal data of another. Data coupling = BEST (simple data passed).

2. Which tool is used for structured designing? [PYQ 2018]

- (i) Program flowchart
- ✓ (ii) Structure chart ← Correct
- (iii) Dataflow diagram
- (iv) Module

Explanation: Structure Chart = PRIMARY tool for structured design. Shows hierarchical module relationships and data/control flows.

3. The context diagram of a DFD is also known as? [PYQ 2019]

- ✓ (i) Level 0 DFD ← Correct
- (ii) Level 1 DFD
- (iii) Level 2 DFD
- (iv) None of the above

Explanation: Context Diagram = Level 0 DFD. Shows entire system as ONE process bubble with all external entities. Broadest view.

4. DFD shows? [PYQ 2018]

- (i) The flow of data
- (ii) The processes
- (iii) The areas where data is stored

✓ (iv) All of the above ← Correct

Explanation: DFD shows ALL: data flows (arrows), processes (ovals), data stores (parallel lines), and external entities (rectangles).

5. In a use case diagram, the stick person icon represents? [PYQ 2019]

- (i) Internal system
- (ii) External system

✓ (iii) Human user ← Correct

- (iv) None of the above

Explanation: Actor (stick person) in UML use case diagram = human user or external system that interacts with the software.

6. Top-Down Design starts with?

- (i) Low-level modules

✓ (ii) High-level system overview ← Correct

- (iii) Individual functions
- (iv) Database design

Explanation: Top-Down = starts with BIG PICTURE (whole system) and progressively decomposes into smaller sub-modules.

7. The process of hiding internal implementation details is called?

- (i) Encapsulation
- ✓ (ii) Abstraction ← Correct
- (iii) Inheritance
- (iv) Polymorphism

Explanation: Abstraction = hiding complex implementation details; exposing only essential features. Key OOD principle.

8. Which is NOT a component of a DFD?

- (i) Process (oval)
- (ii) Data Flow (arrow)
- (iii) Data Store (parallel lines)
- ✓ (iv) Decision Box (diamond) ← Correct

Explanation: Decision box (diamond) belongs to FLOWCHARTS — NOT DFDs. DFD has: Process, Data Flow, Data Store, External Entity.

9. Transform Analysis is used when?

- (i) Multiple transactions of different types exist
- ✓ (ii) Data flows in a clear input-transform-output path ← Correct
- (iii) Requirements are unclear
- (iv) System is event-driven

Explanation: Transform Analysis: data flows linearly — Input → Central Transform → Output. Divides DFD into afferent, central, efferent.

10. Object-Oriented Design primarily uses which notation?

- (i) DFD
- ✓ (ii) UML (Unified Modeling Language) ← Correct
- (iii) Structure charts
- (iv) E-R Diagrams

Explanation: UML = STANDARD notation for OOD. Includes Class Diagrams, Sequence Diagrams, Use Case Diagrams, Activity Diagrams.

SECTION B — 4-Mark Questions | Unit 3

Q1. What do you mean by balancing a DFD? Illustrate with example. [PYQ 2014]

Introduction: Balancing a DFD means ensuring that data flows crossing the boundary of a process in a higher-level DFD are exactly the same as the data flows entering/leaving the lower-level DFD that decomposes that process.

Key Points:

- **Rule:** Every input/output data flow at Level N must appear in the Level N+1 DFD decomposing that process.
- **Example:** If Level 1 Process "Process Order" has inputs: Customer_Order, Payment_Info and outputs: Confirmation, Invoice — then Level 2 DFD (decomposing "Process Order") MUST show the SAME four data flows crossing its boundary.
- **Balanced:** Same data flows appear at both levels. No data mysteriously appears or disappears.
- **Unbalanced:** Level 2 shows extra flows not in Level 1, or missing flows that Level 1 shows.
- **Why Important:** Ensures consistency across DFD hierarchy. Prevents data loss in the system design.

Conclusion: Balanced DFDs ensure hierarchical decomposition is complete and consistent — a fundamental correctness requirement for structured analysis.

Q2. What are the differences between Function-Oriented and Object-Oriented Design? [PYQ 2018, 2019]

Introduction: Two fundamental design paradigms — Function-Oriented focuses on WHAT the system does (functions); Object-Oriented focuses on WHAT ENTITIES the system manages (objects).

Key Points:

- **Basic Unit:** FOD = Function/Procedure | OOD = Object (Data + Behavior combined)
- **Approach:** FOD = Top-Down Decomposition | OOD = Bottom-Up Composition
- **Data Handling:** FOD = Data and functions separate; global data | OOD = Data encapsulated within objects, protected
- **Reusability:** FOD = Limited (functions tied to specific data) | OOD = High (inheritance, composition)
- **Maintainability:** FOD = Harder (global data change affects many functions) | OOD = Easier (changes localized in objects)
- **Notation:** FOD = DFD + Structure Charts | OOD = UML (Class, Sequence, Use Case Diagrams)

Conclusion: FOD suits stable, computation-heavy systems. OOD excels in large, evolving systems requiring maintainability and reusability.

Q3. Describe the concepts of Cohesion and Coupling with suitable examples. [PYQ 2022]

Introduction: Cohesion and Coupling are the two most important design quality metrics. GOAL: Maximize Cohesion + Minimize Coupling = Good Design.

Key Points:

- **Cohesion (WANT HIGH):** Degree to which elements WITHIN a module functionally belong together.
- **High Cohesion Example:** Module "CalculateTax()" does ONLY tax calculation — single responsibility.
- **Low Cohesion Example:** Module does printing + tax calculation + email sending — unrelated tasks lumped together.
- **Cohesion Types (Low to High):** Coincidental → Logical → Temporal → Procedural → Communicational → Sequential → Functional (BEST)
- **Coupling (WANT LOW):** Degree of INTERDEPENDENCE between modules.
- **Low Coupling Example (Data):** Module A passes only a number to Module B — minimal dependency.
- **High Coupling Example (Content — WORST):** Module A directly reads/modifies Module B's internal variables.
- **Coupling Types (Best to Worst):** Data → Stamp → Control → Common → Content (WORST)

Conclusion: Design principle: every module should have a single, well-defined purpose (high cohesion) and interact with other modules through minimal, clean interfaces (low coupling).

Q4. Explain the design steps of Transform Analysis (Transform Mapping). [PYQ 2017, 2020]

Introduction: Transform Analysis converts a Data Flow Diagram (DFD) into a Structure Chart by identifying afferent (input), central transform, and efferent (output) branches.

Key Points:

- **Step 1 — Review DFD:** Understand the complete data flow from input to output.
- **Step 2 — Identify Afferent Branch:** Data flows handling incoming data (reading, validating inputs).
- **Step 3 — Identify Central Transform:** Core processing part — the heart of the computation.
- **Step 4 — Identify Efferent Branch:** Data flows formatting and producing outputs.
- **Step 5 — Create Top Module:** Main controller calls Get_Input, Process_Data, Put_Output modules.
- **Step 6 — Refine:** Decompose each branch into lower modules. Apply high-cohesion, low-coupling principles.

Conclusion: Transform Analysis is most effective for systems with a clear linear data flow pipeline. Transaction Analysis is used when the system handles multiple different types of transactions.

Q5. Explain the levels of DFD (Context Diagram, Level 1, Level 2) with example. [PYQ 2018, 2019]

Introduction: DFD is created hierarchically — each level provides more detail while maintaining balance with the level above. Context → Level 1 → Level 2 → ...

Key Points:

- **DFD Symbols:** Oval = Process (transforms data) | Arrow = Data Flow (data movement) | Rectangle = External Entity (source/sink) | Parallel Lines = Data Store (storage)
- **Level 0 (Context Diagram):** Entire system shown as ONE process bubble. Shows all external entities and their data flows. Highest abstraction — "black box" view. Must be balanced with Level 1.
- **Level 1 DFD:** Expands the single process into major sub-processes. Shows major functions. Shows data stores. All data flows must balance with Level 0.
- **Level 2 DFD:** Further decomposes each Level 1 process. Provides detailed internal view. Continue until each process is simple enough to code directly.

Conclusion: DFD hierarchy manages complexity — start big-picture (Context), progressively zoom in (Level 1, 2...) until each process is small enough to implement.

SECTION C — 12-Mark Questions | Unit 3

Q1. Describe the concept of Software Design. Explain the difference between High-Level Design and Low-Level Design. [PYQ 2024, 2025]

Introduction: Software Design transforms requirements (SRS) into a blueprint for building the software. It bridges the gap between WHAT the system must do (requirements) and HOW it will be built (implementation). Good design minimizes complexity and maximizes maintainability.

Answer:

- **Objectives of Software Design:** Create structure satisfying all requirements; minimize complexity and coupling; maximize cohesion and reusability; produce efficient, maintainable system.
- **High-Level Design (HLD) — Definition:** Also called Architectural Design or System Design. Focuses on WHAT major components exist and HOW they interact at the system level.
- **HLD Content:** System architecture (layers, modules), technology stack selection, major module identification, database schema (conceptual), external API/system integrations, non-functional requirements design (caching, load balancing).
- **HLD Audience & Output:** Architects, senior developers, managers, clients. Output = HLD Document with block diagrams, system architecture diagrams, high-level DFDs.
- **HLD Example:** "The system will have 3 layers: Presentation (React), Business Logic (Node.js), Data Access (MySQL). Authentication via JWT tokens."
- **Low-Level Design (LLD) — Definition:** Also called Detailed Design or Module Design. Focuses on HOW each individual module/function is implemented internally.
- **LLD Content:** Detailed algorithms, data structures, function signatures, class designs, database tables (detailed with columns/types), pseudo-code, error handling logic.
- **LLD Audience & Output:** Individual developers and testers. Output = LLD Document with structure charts, class diagrams, flowcharts, sequence diagrams, E-R diagrams.
- **LLD Example:** "Login function: 1) Validate email regex, 2) Hash password with SHA-256, 3) Query users table WHERE email=? AND pass=?, 4) Return JWT or HTTP 401."

Diagram / Representation:

```
HLD (System Level): LLD (Module Level):
[Presentation] login(email, password):
| REST API 1. validate_email(email)
[Business Logic] 2. hash = SHA256(password)
| DB calls 3. user = db.query(email, hash)
[Data Layer / DB] 4. if user: return JWT_token
5. else: raise AuthError
```

Conclusion: HLD gives the big picture ensuring the right architecture is chosen. LLD provides implementation details ensuring correct module behavior. Both are essential — skipping either leads to costly rework during development or

maintenance.

Q2. Describe Cohesion and Coupling concepts in detail. State differences with suitable examples. [PYQ 2022]

Introduction: Cohesion and Coupling are the two foundational design quality metrics in Function-Oriented Design. Together they determine how maintainable, testable, and modifiable software is. Golden Rule: MAXIMIZE COHESION, MINIMIZE COUPLING.

Answer:

- **COHESION — Detailed Types (Worst to Best):**

- **1. Coincidental (Worst):** Elements grouped randomly with no logical relationship. Example: PrintReport(), CalculateTax(), SendEmail() all in one "Utils" module. Hard to understand, reuse, or maintain.
- **2. Logical:** Related activities grouped; one activated by a flag parameter. Example: All I/O functions in one module, selected by type parameter. Still weak — unrelated code coexists.
- **3. Temporal:** Elements execute at the same time. Example: Initialization module that sets up all variables, opens files, and connects to database — all happening at startup.
- **4. Procedural:** Elements execute in a specific required order. Example: ValidateInput() → FormatData() → StoreRecord() — order matters but functions serve different purposes.
- **5. Communicational:** Elements operate on the same data. Example: All operations on a CustomerRecord object — AddCustomer, UpdateCustomer, DeleteCustomer.
- **6. Sequential:** Output of one element feeds directly into next. Example: ReadFile() output → ParseData() output → ValidateRecord() output → StoreToDatabase().
- **7. Functional (Best):** Module performs one single, well-defined function. Example: CalculateTax(amount, rate) does ONLY tax calculation — nothing else. Maximum reusability and testability.

- **COUPLING — Detailed Types (Best to Worst):**

- **1. Data Coupling (Best):** Only simple data parameters passed. Example: Module A calls CalculateArea(radius) — passes only a number, receives only a number. Minimal dependency.
- **2. Stamp Coupling:** Data structure passed but not all fields used. Example: Passing entire CustomerRecord object when only the Name field is needed.
- **3. Control Coupling:** One module controls flow of another via a flag. Example: ProcessOrder(flag) where flag=1 means new order, flag=2 means update existing. Tight behavioral dependency.
- **4. Common Coupling:** Multiple modules share a global variable. Example: All modules read/write a global variable 'currentUserID'. One module's error corrupts all others.
- **5. Content Coupling (Worst):** Module directly accesses internal data of another. Example: Module A reads/modifies Module B's private variables directly. Maximum coupling — any change to B breaks A.

Diagram / Representation:

Cohesion Scale (Low to High):

Coincidental → Logical → Temporal → Procedural → Communicational → Sequential → Functional
(WORST) (BEST)

Coupling Scale (High to Low coupling i.e. BAD to GOOD):

Content → Common → Control → Stamp → Data
(WORST) (BEST)

Conclusion: High cohesion makes modules easier to understand, test, and reuse. Low coupling makes changes safer and the system more maintainable. These two properties are the foundation of SOLID principles in OOP and Separation of Concerns in architecture.

Q3. Which symbols are used for designing DFDs? Explain with example DFD. Explain synchronous and asynchronous dataflow. [PYQ 2018]

Introduction: A Data Flow Diagram (DFD) graphically represents how data flows through a system — what is processed, stored, and transferred — without any implementation details. It is the primary tool in Structured Analysis.

Answer:

- **1. Process (Oval/Circle):** Transforms input data into output. Named with verb phrase (e.g., "Validate Order"). Has unique number (1.0, 2.0). Represents a system function.

- **2. Data Flow (Arrow):** Movement of data between components. Labeled with data name (e.g., "Customer_Order"). Direction shows flow. Must be named — never unnamed.
- **3. Data Store (Parallel Lines):** Persistent storage (database table, file). Named as noun (e.g., "D1: Customer Records"). Processes read from and write to stores.
- **4. External Entity (Rectangle):** Source or sink of data OUTSIDE system boundary. Cannot be changed by the system. Examples: Customer, Bank, Supplier, Government.
- **Synchronous Dataflow:** Processes operate at the same rate in real-time. Data flows continuously without buffering. Example: Real-time sensor → controller → actuator in embedded system. Shown as direct arrows between processes.
- **Asynchronous Dataflow:** Processes operate independently at different rates. Data stored in a buffer/data store between processes. Example: Order placed (stored in DB) → later processed by billing system. Shown as data store between processes.

Diagram / Representation:

Example DFD – Library System (Level 1):

```
[Student] --{Book Request}--> (1.0 Issue Book) --{Receipt}--> [Student]
| |
[Read] [Write]
| |
[D1: Book DB] [D2: Issue Records]

[Student] --{Return Request}--> (2.0 Return Book) --> [D1: Book DB]
|
(3.0 Calculate Fine) --> [D2: Issue Records]
|
{Fine Amount} --> [Student]
```

Conclusion: DFDs are powerful communication tools — they convey system behavior to non-technical stakeholders and form the foundation for structured design (Structure Charts). The hierarchical nature allows managing complexity from big picture to implementation details.

UNIT 4: Coding, Testing + Software Reliability

Unit Testing · Black Box · White Box · Debugging · Program Analysis · System Testing · Musa's Model

SECTION A — MCQ (2 Marks Each) | Unit 4

1. The main purpose of integration testing is to find? [PYQ 2014]

- (i) Design errors
- (ii) Analysis errors
- (iii) Procedure errors
- ✓ (iv) Interface errors ← Correct

Explanation: Integration testing detects INTERFACE errors — problems in communication and interaction between modules.

2. When all modules are integrated and tested as a complete system at once, it is called? [PYQ 2014]

- (i) Bottom-up testing
- ✓ (ii) Big-bang testing ← Correct
- (iii) Top-down testing
- (iv) Sandwich testing

Explanation: Big-bang testing = ALL modules integrated simultaneously and tested together. Simple but hard to isolate bugs.

3. Black box testing is also called?

- ✓ (i) Functional testing ← Correct
- (ii) Structural testing
- (iii) Glass-box testing
- (iv) White-box testing

Explanation: Black-box = Functional/Behavioral testing. Tests WHAT system does (inputs/outputs) without knowing internal code.

4. White-box testing is primarily performed by?

- ✓ (i) Developers ← Correct
- (ii) End users
- (iii) Business analysts
- (iv) Project managers

Explanation: White-box testing requires full knowledge of code — primarily done by DEVELOPERS who wrote the code.

5. Which is a BLACK-BOX testing technique? [PYQ 2022]

- (i) Statement Coverage
- ✓ (ii) Equivalence Partitioning ← Correct
- (iii) Path Coverage
- (iv) Branch Coverage

Explanation: Equivalence Partitioning = Black-box technique. Statement/Branch/Path coverage = White-box techniques.

6. Regression testing is performed? [PYQ 2014]

- (i) Before coding begins
- (ii) After requirements gathering only
- ✓ (iii) After bug fixes to ensure no new bugs introduced ← Correct

- (iv) Only at project completion

Explanation: Regression testing re-runs existing tests after any change to ensure the change did not break other features.

7. Musa's Basic Model is used for? [PYQ 2014,2024]

✓ (i) Software reliability estimation ← Correct

- (ii) Cost estimation
(iii) Design validation
(iv) Requirements analysis

Explanation: Musa's Basic Execution Time Model (1975) estimates software reliability — specifically failure rate vs execution time.

8. Which debugging strategy traces backward from the failure point?

- (i) Brute Force
✓ (ii) Backtracking ← Correct
(iii) Forward Analysis
(iv) Static Analysis

Explanation: Backtracking = trace program execution BACKWARD from the failure point until the root cause is identified.

9. Static Code Analysis involves?

- (i) Executing the program with test inputs
✓ (ii) Analyzing code WITHOUT executing it ← Correct
(iii) Running performance benchmarks
(iv) Unit testing modules

Explanation: Static Analysis = examining source code WITHOUT executing it to find bugs, vulnerabilities, and style violations.

10. Statement coverage ensures?

- ✓ (i) Every executable statement is executed at least once ← Correct
(ii) Every branch is tested
(iii) All paths are tested
(iv) Every loop runs 10 times

Explanation: Statement Coverage = simplest white-box criterion. Every executable statement must run at least once during testing.

SECTION B — 4-Mark Questions | Unit 4

Q1. What is the difference between Black-box Testing and White-box Testing? [PYQ 2014, 2017, 2019]

Introduction: These two approaches represent fundamentally different perspectives on testing — external behavior (black-box) versus internal structure (white-box). Both are essential for comprehensive testing.

Key Points:

- **Black-Box Testing:** Also called Functional/Behavioral Testing. Tests WHAT the system does (inputs → outputs) without any knowledge of internal code.
- **White-Box Testing:** Also called Structural/Glass-box Testing. Tests HOW the system implements functionality — requires full knowledge of source code.
- **Knowledge Required:** Black-box = None | White-box = Full code access needed.
- **Performed By:** Black-box = Testers/QA team | White-box = Developers.
- **Techniques:** Black-box = Equivalence Partitioning, BVA, Decision Tables | White-box = Statement, Branch, Path Coverage.
- **Test Basis:** Black-box = SRS/Requirements | White-box = Source code, design documents.
- **Used In:** Black-box = System/Acceptance Testing | White-box = Unit/Integration Testing.

Conclusion: Both are essential — White-box ensures internal code is correct; Black-box ensures system requirements are met from the user's perspective. Use both for maximum coverage.

Q2. What is regression testing? When is it done? Why is it necessary? [PYQ 2014]

Introduction: Regression Testing is re-executing existing test cases after any code change (bug fix, new feature, refactoring) to ensure the change did not break previously working functionality.

Key Points:

- **When Done:** After fixing any bug — verify fix works AND nothing else broke.
- **When Done:** After adding new features — ensure new code doesn't affect existing features.
- **When Done:** After code refactoring — verify behavior unchanged after code cleanup.
- **When Done:** After every build in CI/CD pipelines — automated regression ensures continuous quality.
- **Why Necessary:** Software changes create unexpected ripple effects — a fix in Module A can break Module B.
- **Why Necessary:** Prevents "regression defects" — bugs that reappear after previously being fixed.

Conclusion: Regression testing is the safety net of software development. Without it, every bug fix or enhancement risks breaking existing working functionality. Modern CI/CD pipelines run automated regression suites on every code commit.

Q3. What are Test Oracles? Explain with examples. [PYQ 2017, 2020]

Introduction: A Test Oracle is a mechanism used to determine whether the output of a test execution is CORRECT — it provides the expected result to compare against the actual result.

Key Points:

- **Specification Oracle:** Expected output derived from SRS/requirements. Example: "FR-05 states login with valid credentials returns HTTP 200 and auth token."
- **Existing System Oracle:** Previous version of system used as reference. Compare new system output with legacy system output for same inputs.
- **Human Oracle:** Domain expert manually verifies whether output is correct for complex, hard-to-specify outputs.
- **Statistical Oracle:** Uses statistical sampling to determine expected output range (useful for AI/ML systems).
- **The Oracle Problem:** For some systems (AI, search engines, creative tools), it is difficult or impossible to pre-define the correct expected output.

Conclusion: Test Oracles are essential for automated testing — without them, you can execute tests but cannot automatically determine pass/fail. Defining a reliable oracle is often the hardest part of test design.

Q4. Explain Boundary Value Analysis (BVA). Give an example. [PYQ 2022]

Introduction: Boundary Value Analysis (BVA) is a black-box test design technique based on the observation that ERRORS TEND TO CLUSTER AT THE BOUNDARIES of input domains rather than in the middle.

Key Points:

- **Why Boundaries:** Off-by-one errors ($>$ vs $\geq$) are extremely common. Boundary values expose fence-post errors, loop termination errors, range check mistakes.
- **BVA Rules for input range $[a, b]$:** Test $a-1$ (just below minimum — invalid), a (minimum — boundary), $a+1$ (just above minimum — valid), middle value (nominal), $b-1$ (valid), b (maximum — boundary), $b+1$ (just above maximum — invalid).
- **Example — Age input field (valid range 18 to 60):** Test: 17 (invalid), 18 (min boundary), 19 (valid), 38 (nominal), 59 (valid), 60 (max boundary), 61 (invalid).
- **Complements Equivalence Partitioning:** EP reduces test cases by grouping; BVA focuses on critical boundary points. Use BOTH together for effective testing.

Conclusion: BVA is one of the most cost-effective testing techniques — boundaries are where most bugs hide. Combining BVA with Equivalence Partitioning gives maximum defect detection with minimum test cases.

Q5. What is Software Quality Assurance (SQA)? What activities does it include? [PYQ 2020]

Introduction: Software Quality Assurance (SQA) is a systematic, planned set of activities ensuring that software development processes and products conform to defined quality standards. SQA is PROACTIVE — prevents defects rather than just finding them.

Key Points:

- **Standards Application:** Define and enforce coding standards, design standards, documentation standards.
- **Technical Reviews:** Formal inspections, walkthroughs, peer reviews of requirements, design, and code.
- **Multi-Level Testing Strategy:** Unit → Integration → System → Acceptance testing.
- **Defect Tracking:** Systematic logging, tracking, and resolution of all discovered defects.
- **Configuration Management:** Version control, change management, baseline management.
- **Process Audits:** Verifying that defined development processes are actually being followed.

Conclusion: SQA is built into EVERY phase of the SDLC — not just the end. The principle is "build quality IN, not test it IN." The cost of SQA is always less than the cost of poor quality discovered in production.

SECTION C — 12-Mark Questions | Unit 4

Q1. Explain Integration Testing approaches in detail. [PYQ 2018]

Introduction: Integration Testing combines individually tested modules and tests them as a group. Its purpose is to expose defects in INTERFACES and INTERACTION between integrated components that unit testing cannot find.

Answer:

- **Why Needed:** Unit tests verify modules in isolation. When modules interact, new bugs emerge: data format mismatches, wrong API calls, incorrect assumptions about shared data, timing issues.
- **Approach 1 — Big Bang Integration:** All modules integrated simultaneously and tested together. Advantage: Simple, quick to start. Disadvantage: Very hard to isolate which interface caused which bug. Suitable for very small systems only.
- **Approach 2 — Top-Down Integration:** Start from TOP (main control) module. Integrate downward using STUBS (dummy replacements for lower unimplemented modules). Advantage: Tests high-level logic early; can demo partial system to client. Disadvantage: Critical lower modules tested late; many stubs needed.
- **Approach 3 — Bottom-Up Integration:** Start from LOWEST-level modules. Integrate upward using DRIVERS (programs that call the module being tested). Advantage: Real modules tested early; no stubs needed. Disadvantage: High-level control logic tested very late; no early client demo.
- **Approach 4 — Sandwich/Hybrid Integration:** Combines Top-Down and Bottom-Up simultaneously. Top levels use stubs; bottom levels use drivers; meet in the middle. Advantage: Balances both approaches; reduces overall stubs and drivers needed.

Diagram / Representation:

```
Top-Down: [Main] --(stub)--> [Module A] --(stub)--> [Module B]
(test Main first with stubs for A and B)

Bottom-Up: [Module C] --(driver)--> [Module B] --(driver)--> [Main]
(test C first using driver, then integrate upward)

Sandwich: Top-Down from top + Bottom-Up from bottom → meet in middle
```

Conclusion: Integration testing is often the most complex phase. Choose approach based on system structure: Top-Down for control-heavy systems, Bottom-Up for utility/algorithm-heavy systems, Sandwich for large complex systems where critical functionality is distributed across both top and bottom modules.

Q2. Explain Black Box Testing methods and their advantages and disadvantages. [PYQ 2022]

Introduction: Black-box testing (Functional/Behavioral Testing) evaluates software behavior WITHOUT knowledge of internal code. Testers interact as end-users would — providing inputs and verifying outputs against requirements.

Answer:

- **Technique 1 — Equivalence Partitioning:** Divides input domain into equivalence classes. Assumption: all values in a class behave identically. Test ONE value from each partition. Example: Age [18-60]: test invalid-low (<18), valid (18-60), invalid-high (>60). Advantage: Reduces test cases dramatically.
- **Technique 2 — Boundary Value Analysis:** Tests values AT and AROUND boundaries. Example: Age [18-60]: test 17, 18, 19, 59, 60, 61. Advantage: High defect detection (most errors occur at boundaries).
- **Technique 3 — Decision Table Testing:** Tests all combinations of conditions and actions. Creates table: rows=conditions, columns=test cases, cells=T/F. Advantage: Covers all condition combinations; ideal for business rules.
- **Technique 4 — State Transition Testing:** Tests behavior as system transitions between states. Example: ATM states: Idle → Card Inserted → PIN Entered → Authenticated → Transaction. Advantage: Good for event-driven, menu-driven systems.
- **Advantages of Black-Box Testing:** Tests from user perspective; no code knowledge needed; unbiased (testers don't know internal code); suitable for system and acceptance testing.
- **Disadvantages of Black-Box Testing:** Cannot ensure all code paths are tested; may miss internal algorithm errors; test cases can be redundant if poorly designed.

Diagram / Representation:

```

Black-Box Test Design:
SRS / Requirements
|
[Test Design]
/ | \ \
EP BVA DT STT
| | | |
Test Cases Generated
|
[Execute Tests]
|
[Compare Actual vs Expected Output via TEST ORACLE]

```

Conclusion: Black-box testing is essential for validating user requirements. It complements white-box testing — use both for comprehensive coverage. In practice, black-box techniques are used for system testing and UAT, while white-box is used during unit and integration testing.

Q3. Explain software testing importance and describe different levels of software testing. [PYQ 2024]

Introduction: Software testing is the process of evaluating a software application to find defects and verify it meets requirements. It builds confidence that software works correctly and is fit for deployment. Cost of defects increases 10-100x when found in production vs during development.

Answer:

- **Level 1 — Unit Testing:** Tests individual functions, methods, or modules in ISOLATION. Performed by developers during coding. Technique: White-box testing. Goal: Verify each unit produces correct output for given inputs. Example: Test CalculateTax(amount, rate) returns correct values. Uses stubs/mocks for dependencies.
- **Level 2 — Integration Testing:** Tests how multiple units work TOGETHER. Detects interface defects (data format mismatches, wrong API calls). Approaches: Top-Down, Bottom-Up, Big-Bang, Sandwich. Goal: Verify modules interact correctly through their interfaces.
- **Level 3 — System Testing:** Tests the COMPLETE integrated system against ALL requirements. Performed by dedicated test team using black-box approach. Types include: Functional, Performance, Load, Stress, Security, Usability, Compatibility testing. Goal: Verify entire system meets both functional and non-functional requirements.
- **Level 4 — Acceptance Testing:** Verifies system meets BUSINESS requirements and is ready for deployment. Performed by client/end users. Types: User Acceptance Testing (UAT), Alpha testing (in-house by end users), Beta testing (limited external release). Goal: Confirm system is acceptable to the paying customer.

Diagram / Representation:

```

Testing Pyramid:
[UAT / Acceptance] <- Few, high value, expensive
[ System Testing ] <- Moderate tests
[ Integration Testing ] <- More tests
[ Unit Testing ] <- Many, fast, cheap (most tests here)

```

Rule: Most tests at bottom (unit), fewest at top (acceptance)

Conclusion: A balanced testing strategy across all four levels is essential. Each level serves a distinct purpose. Testing at lower levels is cheaper and catches issues earlier — the earlier a bug is found, the cheaper it is to fix. Invest heavily in unit testing to prevent integration and system-level issues.

UNIT 5: Software Quality Assurance + Maintenance

ISO 9000 · SEI CMM · Maintenance Process Models · Reverse Engineering · Maintenance Cost Estimation

SECTION A — MCQ (2 Marks Each) | Unit 5

1. ISO 9000 is a set of international standards for? [PYQ 2024,2025]

- (i) Software testing procedures
- ✓ (ii) Quality management systems ← Correct
- (iii) Project scheduling techniques
- (iv) Cost estimation methods

Explanation: ISO 9000 = International standard for Quality Management Systems ensuring consistent processes deliver consistent quality.

2. SEI CMM has how many maturity levels? [PYQ 2018]

- (i) 3 levels
- (ii) 4 levels
- ✓ (iii) 5 levels ← Correct
- (iv) 6 levels

Explanation: SEI CMM = 5 levels: Initial → Repeatable → Defined → Managed → Optimizing. Level 5 = highest maturity.

3. Which type of maintenance fixes bugs DISCOVERED AFTER software delivery? [PYQ 2020]

- (i) Perfective
- (ii) Adaptive
- ✓ (iii) Corrective ← Correct
- (iv) Preventive

Explanation: Corrective maintenance = fixing defects found by users after software is deployed. Most urgent type.

4. Change made to system to REDUCE FUTURE SYSTEM FAILURE is called? [PYQ 2014]

- (i) Corrective maintenance
- (ii) Adaptive maintenance
- (iii) Perfective maintenance
- ✓ (iv) Preventive maintenance ← Correct

Explanation: Preventive maintenance = restructuring code to prevent future failures. Also called Software Re-engineering.

5. Reverse Engineering in software means? [PYQ 2024,2025]

- (i) Building software from scratch
- (ii) Writing code backward
- ✓ (iii) Analyzing existing software to recover its design ← Correct
- (iv) Removing unwanted software features

Explanation: Reverse Engineering = analyzing existing software to understand design, recover lost documentation, aid migration.

6. The HIGHEST level in SEI CMM is? [PYQ 2018]

- (i) Level 3: Defined
- (ii) Level 4: Managed

✓ (iii) Level 5: Optimizing ← Correct

(iv) Level 5: Controlled

Explanation: CMM Level 5: Optimizing = continuous process improvement using quantitative feedback. Highest maturity.

7. Which maintenance type adapts software to a NEW ENVIRONMENT? [PYQ 2020]

(i) Corrective

✓ (ii) Adaptive ← Correct

(iii) Perfective

(iv) Preventive

Explanation: Adaptive maintenance = modifying software for new OS, hardware, database, or regulatory environment.

8. SEI CMM was developed by?

✓ (i) Carnegie Mellon University / SEI ← Correct

(ii) IEEE

(iii) ISO

(iv) Barry Boehm

Explanation: SEI CMM developed at Carnegie Mellon University by Watts Humphrey for US Department of Defense (1991).

9. ISO 9000 primarily focuses on?

(i) Technical coding standards

✓ (ii) Processes and quality management procedures ← Correct

(iii) Network configuration

(iv) Cost reduction programs

Explanation: ISO 9000 focuses on PROCESSES — defining, documenting, and following quality management processes.

10. Which type of maintenance adds NEW FEATURES or improves performance?

(i) Corrective

(ii) Adaptive

✓ (iii) Perfective ← Correct

(iv) Preventive

Explanation: Perfective maintenance = adding new features, improving performance, enhancing usability. Accounts for ~50% of all maintenance!

SECTION B — 4-Mark Questions | Unit 5

Q1. Explain the SEI CMM (Capability Maturity Model) levels. [PYQ 2018]

Introduction: SEI CMM (Software Engineering Institute Capability Maturity Model) defines 5 levels of software process maturity — from chaotic (Level 1) to continuously improving (Level 5).

Key Points:

- **Level 1 — Initial:** Unpredictable, chaotic. Success depends on individual heroics. No formal processes. Example: Startup with ad-hoc development. Most organizations start here.
- **Level 2 — Repeatable:** Basic project management (schedule, cost, requirements tracking). Projects can repeat past successes. Processes documented per-project.
- **Level 3 — Defined:** Organizational STANDARD processes documented and tailored per project. Training programs exist. Most mature Indian IT companies operate here.
- **Level 4 — Managed:** Quantitative management — processes MEASURED and controlled using statistical data. Predictable, controllable performance.

- **Level 5 — Optimizing:** CONTINUOUS process improvement using quantitative feedback. Proactive defect prevention. Example: NASA, top defense contractors. Highest maturity.

Conclusion: Each CMM level builds on the previous. Moving from Level 1 to Level 3 alone can reduce defect rates by 50-75% and improve schedule predictability. Higher CMM levels correlate directly with project success rates.

Q2. What is software maintenance? What are the categories of software maintenance? [PYQ 2020]

Introduction: Software Maintenance is the process of modifying software AFTER delivery to correct faults, improve performance, or adapt to a changed environment. It is typically the longest and most expensive phase (60-80% of total lifecycle cost).

Key Points:

- **1. Corrective Maintenance (~21%):** Fixing defects discovered after deployment. User reports bug → investigate → fix → test → release patch. Most urgent type.
- **2. Adaptive Maintenance (~25%):** Modifying software for new environment. Examples: New OS, new database version, new hardware, regulatory changes (GDPR compliance).
- **3. Perfective Maintenance (~50% — LARGEST):** Adding new features or improving existing ones. Performance optimization, code cleanup, UI improvements. Most of maintenance is actually enhancement!
- **4. Preventive Maintenance (~4%):** Making changes to PREVENT future failures. Restructuring aging code, updating documentation, improving maintainability. Also called re-engineering.

Conclusion: Perfective maintenance accounts for ~50% of all maintenance work — most maintenance is enhancements, not bug fixes. Planning for maintainability during design dramatically reduces long-term maintenance costs.

Q3. What is the significance of Reverse Engineering in software? [PYQ 2024, 2025]

Introduction: Reverse Engineering (RE) in software is the process of analyzing an existing system to identify its components, their relationships, and design — essentially recovering design from implementation when documentation is lost.

Key Points:

- **Legacy System Recovery:** Old systems with lost documentation. RE recovers design documents, helping maintenance teams understand what the system does and how.
- **Migration Aid:** Before migrating legacy system to new platform, RE creates a complete picture of the current system architecture and functionality.
- **Security Analysis:** Security researchers use RE to find vulnerabilities in software (malware analysis, penetration testing, vulnerability research).
- **Re-engineering First Step:** You must understand what EXISTS before redesigning. RE is the mandatory first step in software re-engineering.
- **Typical RE Activities:** Redocumentation (generate docs from code) → Design Recovery (recreate high-level design) → Process Recovery (understand business logic).

Conclusion: Reverse Engineering is increasingly critical as organizations struggle to maintain aging legacy systems that were never properly documented. It is the first step in modernizing legacy software and cannot be skipped.

Q4. Discuss the concept of estimation of software maintenance costs. [PYQ 2022, 2024, 2025]

Introduction: Estimating maintenance costs is critical for project planning. Maintenance typically consumes 60-80% of total lifecycle costs and often exceeds the original development cost.

Key Points:

- **COCOMO Maintenance Formula:** Maintenance Effort = SDE x ACT, where SDE = original Software Development Effort, ACT = Annual Change Traffic (fraction changed per year).
- **Cost Factors (Boehm):** Application type, staff familiarity, code age, external environment stability, code quality (poor design = exponentially higher maintenance cost).
- **Maintenance Cost Distribution:** Corrective 21%, Adaptive 25%, Perfective 50%, Preventive 4%.
- **Lehman's Law:** Software must continually adapt or become progressively less useful. Complexity increases with each change unless actively managed.

- **Cost Reduction Strategies:** Invest in code quality during development (reduces maintenance 3-5x), maintain documentation, use automated regression testing, apply regular refactoring.

Conclusion: The best way to reduce maintenance costs is to build quality IN during development — good design (high cohesion, low coupling), thorough documentation, and comprehensive test suites dramatically reduce long-term maintenance expenditure.

Q5. Explain the difference between ISO 9000 and SEI CMM quality standards. [PYQ 2022, 2024, 2025]

Introduction: Both ISO 9000 and SEI CMM are quality frameworks but differ significantly in origin, scope, approach, and implementation. They complement rather than compete with each other.

Key Points:

- **ISO 9000:** International standard (ISO committee). Any industry — manufacturing, services, IT. PASS/FAIL audit by external auditor. "Document what you do, do what you document." Baseline quality management.
- **SEI CMM:** Software-specific framework (Carnegie Mellon / SEI). Software organizations only. Level 1-5 RATING by SEI-certified assessor. Staged improvement path. Continuous maturity growth.
- **Key Difference:** ISO 9000 = minimum quality management baseline | CMM = staged roadmap for CONTINUOUS improvement.
- **Scope:** ISO = all industries | CMM = software development organizations only.
- **Result:** ISO = ISO 9001 Certificate | CMM = CMM Level N Rating (1-5).
- **Relationship:** CMM Level 3 organization typically satisfies all ISO 9001 requirements — CMM goes BEYOND ISO 9000.

Conclusion: Organizations often pursue ISO 9001 first (establishes baseline), then pursue higher CMM levels for software-specific excellence. Both demonstrate process quality commitment but serve different audiences and purposes.

SECTION C — 12-Mark Questions | Unit 5

Q1. Compare ISO 9000 and SEI CMM. How do they differ in their approach to software quality? [PYQ 2018, 2022]

Introduction: Both ISO 9000 and SEI CMM aim to improve software quality but through entirely different philosophies, scopes, and mechanisms. Understanding both is essential for software quality management.

Answer:

- **ISO 9000 — Origin & Nature:** International Organization for Standardization. Originally for manufacturing (ISO 9001:1987), adapted for software (ISO 9001:2015). International standard for Quality Management Systems (QMS).
- **ISO 9000 — Philosophy:** "Document what you do, do what you document, prove it." Emphasis on having documented processes and following them consistently. Any industry can apply.
- **ISO 9000 — Assessment:** External auditor performs a PASS/FAIL compliance audit against ISO 9001 requirements. Organization receives ISO 9001 certification if compliant.
- **ISO 9000 — Key Process Areas:** Customer focus, leadership, engagement of people, process approach, improvement, evidence-based decision making, relationship management.
- **ISO 9000 — Result:** ISO 9001 Certificate. Demonstrates minimum quality management standards to customers and regulators. Widely accepted internationally.
- **SEI CMM — Origin & Nature:** Software Engineering Institute (SEI) at Carnegie Mellon University. Developed by Watts Humphrey for US DoD (1991). Software-organization-specific process improvement FRAMEWORK.
- **SEI CMM — Philosophy:** Staged improvement path from chaotic (Level 1) to continuously optimizing (Level 5). Each level builds on previous. Focus on software-specific process maturity.
- **SEI CMM — Assessment:** Certified SEI assessors rate organization on 1-5 maturity scale. Each level has defined Key Process Areas (KPAs) that must be satisfied.
- **SEI CMM Level Details:** L1 Initial (chaotic, heroic) | L2 Repeatable (project mgmt basics) | L3 Defined (org standard processes) | L4 Managed (quantitative metrics) | L5 Optimizing (continuous improvement).
- **Relationship Between ISO and CMM:** CMM Level 3 organization typically satisfies ISO 9001 requirements. CMM goes further — it is a SUPERSET of ISO quality requirements for software organizations.

Diagram / Representation:

Comparison Table:

Feature	ISO 9000	SEI CMM
Origin	ISO (International)	Carnegie Mellon/SEI
Focus	Quality processes	Process maturity levels
Approach	Minimum standards	Continuous improvement
Assessment	Pass/Fail Audit	Level 1-5 Rating
Scope	ANY industry	Software orgs only
Flexibility	More flexible	Prescriptive (KPAs)
Result	ISO 9001 Certificate	CMM Level N Rating
Relationship	CMM L3 satisfies ISO	CMM SUPERSET of ISO

Conclusion: ISO 9000 provides a universal quality management baseline. CMM provides a software-specific roadmap for excellence. Organizations ideally pursue both: ISO 9001 for external recognition, CMM Level 3+ for internal software process maturity. They complement each other rather than compete.

Q2. Discuss in detail the Software Maintenance Process Models. [PYQ 2018, 2019]

Introduction: Software maintenance process models define how maintenance activities are organized and executed. Since maintenance constitutes 60-80% of total software lifecycle cost, structured models are essential for controlling quality, cost, and schedule.

Answer:

- **Why Process Models Needed:** Without a process, maintenance becomes chaotic "firefighting." A structured process ensures: changes are analyzed before implementation, regression risk is managed, documentation stays current, all changes are tested properly.
- **Model 1 — Quick Fix Model:** Bug reported → developer fixes as quickly as possible → deploy without formal process. Advantage: Fast response for critical production issues. Disadvantage: Quick fixes create new bugs, accumulate technical debt, no documentation update. Suitable only for true emergencies.
- **Model 2 — Iterative Enhancement Model:** Treats maintenance as small planned releases. Cycle: Understand current system → Plan change → Modify → Test → Release. Similar to Agile for maintenance. Advantage: Structured, releases frequent improvements reliably. Disadvantage: Needs good test infrastructure and release management.
- **Model 3 — Full Reuse Model:** Before any change, check if existing components can be reused or adapted. Focuses on component repositories and reuse. Advantage: Reduces redundant code, improves consistency, faster changes. Disadvantage: Requires component repository infrastructure.
- **Model 4 — Boehm's Economic Model:** Each proposed change analyzed for expected benefit / maintenance cost ratio. Highest ROI changes prioritized. Advantage: Economic justification for maintenance investment decisions.
- **IEEE 1219 Standard Process (Most Important):** 7 steps: (1) Problem Identification, (2) Analysis (impact, feasibility, cost), (3) Design, (4) Implementation, (5) System Testing, (6) Acceptance Testing, (7) Delivery. Each step has defined inputs, outputs, controls, required resources.
- **Re-Engineering Model:** When maintenance becomes too expensive due to aging code, full re-engineering is needed. Steps: Reverse Engineer (understand existing) → Restructure (improve design) → Forward Engineer (rebuild with modern architecture). Most expensive but resets the maintenance cost curve.

Diagram / Representation:

IEEE 1219 Maintenance Process:

```
Problem Report
|
[Analysis] (impact assessment, feasibility, cost estimate)
|
[Design] (design the change, update docs)
|
[Implement] (code the change, follow standards)
|
[System Test] (unit + integration + regression testing)
|
[Acceptance] (client verifies change meets requirements)
|
[Delivery] (release, update all documentation)
|
(loop back to Problem Report for next change)
```

Conclusion: No single maintenance model fits all situations. Quick Fix for true emergencies; IEEE 1219 for standard planned changes; Iterative Enhancement for regular release cadence; Re-Engineering when legacy code becomes unmaintainable. The key is having ANY defined, repeatable process rather than ad-hoc firefighting.

Q3. Discuss Software Quality Assurance. Discuss one popular maintenance process model. [PYQ 2024, 2025]

Introduction: Software Quality Assurance (SQA) is a systematic, organization-wide set of activities ensuring software products and the processes used to build them meet defined quality standards throughout the entire SDLC. SQA is proactive — it prevents defects.

Answer:

- **SQA vs Software Testing:** Testing finds defects in the product. SQA ensures the PROCESS is followed correctly so defects are less likely to be introduced in the first place.
- **SQA Framework Components:** Standards (coding, documentation, design), Reviews (technical reviews of requirements, design, code), Testing (multi-level strategy), Defect tracking (systematic bug lifecycle management), Configuration Management (version control, change control), Process Audits (verify processes are followed).
- **SQA Plan Contents:** Purpose and scope, list of SQA activities for each phase, standards to be followed, review and audit procedures, defect tracking approach, metrics to be collected.
- **SQA Activities per SDLC Phase:** Requirements = SRS review for correctness and completeness. Design = design walkthroughs and architecture review. Coding = code reviews, static analysis, coding standard compliance checks. Testing = test plan review, test case adequacy assessment. Maintenance = impact analysis before changes.
- **Popular Maintenance Process Model — IEEE 1219:** Step 1: Problem Identification — user reports issue, logged in change management system. Step 2: Analysis — feasibility, impact analysis (what else could break?), cost estimate. Step 3: Design — plan the change, identify affected modules. Step 4: Implementation — code the change per coding standards. Step 5: System Testing — unit test, regression test, integration test. Step 6: Acceptance Testing — client verifies change meets requirements. Step 7: Delivery — release, close change request, update all documentation.
- **Metrics in SQA:** Defect Density (defects/KLOC), Defect Removal Efficiency = defects found before delivery / total defects, Test Coverage %, Review Effectiveness.

Diagram / Representation:

```
SQA Activities across SDLC:
Requirements → Design → Code → Test → Deploy → Maintain
|   |   |   |   |   |
[SRS Review] [Design] [Code] [Test Plan] [Deploy] [Change
[Review] [Review][Review] [Audit] [Control]
```

Conclusion: SQA and structured maintenance processes together ensure software remains reliable, maintainable, and aligned with user needs throughout its operational life. The cost of SQA is always significantly less than the cost of poor quality — defects found early cost 10-100x less to fix than those found in production.

UNIT : Most Repeated Topics (All PYQs 2014–2025)

Topics appearing 5+ times are marked ★VVI — these are guaranteed to appear.

Topic	Unit	Times	VVI?	PYQ Years
Waterfall Model	U1	7x	★VVI	2014,2017,2018,2019,2020,2024,2025
Black Box vs White Box Testing	U4	6x	★VVI	2014,2017,2018,2019,2022,2024
SRS Document	U2	6x	★VVI	2018,2019,2020,2022,2024,2025
COCOMO Model	U2	5x	★VVI	2014,2018,2020,2022,2025
DFD (Levels, Symbols, Balancing)	U3	5x	★VVI	2014,2018,2019,2020,2025
Spiral Model	U1	5x	★VVI	2014,2017,2019,2020,2022
ISO 9000 vs SEI CMM	U5	4x	★	2018,2022,2024,2025
Software Maintenance (Categories)	U5	4x	★	2018,2019,2022,2024
Prototype Model	U1	4x	★	2014,2017,2019,2022
LOC and FP Metrics	U2	4x	★	2014,2018,2020,2022
Coupling & Cohesion	U3	3x	-	2014,2019,2022
Software Reliability / Musa Model	U4	3x	-	2014,2020,2024
Functional vs Non-Functional Req	U2	3x	-	2019,2024,2025
Transform Mapping	U3	2x	-	2017,2020
Software Design (HLD vs LLD)	U3	2x	-	2024,2025

UNIT : Quick Cheat Sheet — Study This the Night Before Exam!

FULL FORMS

- SDLC = Software Development Life Cycle
- COCOMO = COConstructive COSt MOdel
- SRS = Software Requirements Specification
- DFD = Data Flow Diagram
- LOC = Lines Of Code | KLOC = Kilo LOC (1000 LOC)
- FP = Function Point
- CMM = Capability Maturity Model
- SEI = Software Engineering Institute
- MTTF = Mean Time To Failure
- HLD = High Level Design | LLD = Low Level Design
- SQA = Software Quality Assurance
- OOD = Object-Oriented Design | FOD = Function-Oriented Design
- BVA = Boundary Value Analysis | EP = Equivalence Partitioning
- ACT = Annual Change Traffic (in maintenance cost)

KEY DEFINITIONS

- SE: Systematic, disciplined, measurable approach to software development
- Prototype: Preliminary working model for user feedback
- Coupling: Degree of interdependence between modules (WANT LOW)
- Cohesion: Degree to which module elements belong together (WANT HIGH)
- Regression Testing: Re-testing after any change to ensure no new bugs
- Reverse Engineering: Analyzing existing software to recover design
- SRS: Document specifying ALL requirements of a software system
- Corrective Maintenance: Fixing bugs discovered after delivery
- Adaptive Maintenance: Adapting software to new environment
- Perfective Maintenance: Adding features / improving performance (~50% of all maint)

MEMORY TRICKS

- CMM Levels: "India Really Does Make Outstanding software"
- → Initial, Repeatable, Defined, Managed, Optimizing
- Waterfall Phases: "Real Developers Invent Tests During Maintenance"
- → Requirements, Design, Implementation, Testing, Deployment, Maintenance
- DFD Components: "People Drive Data Everywhere"
- → Process, Data Flow, Data Store, External Entity
- Coupling (best to worst): "Data Stamps Control Common Content"
- → Data, Stamp, Control, Common, Content(WORST)
- SRS Chars: "CU3CV" → Correct, Unambiguous, Complete, Consistent, Concise, Verifiable
- Maintenance Types: "CAPP" → Corrective, Adaptive, Perfective, Preventive
- COCOMO: "One Smart Engineer" → Organic, Semidetached, Embedded

COMMON EXAM MISTAKES

- Content coupling = WORST (NOT data coupling). Data coupling = BEST
- Waterfall NOT suitable for "changing requirements" not "small projects"
- SRS NOT characteristic = "Ambiguous" (NOT "Traceable")
- Spiral dimensions = RADIAL + ANGULAR (not diagonal/perpendicular)

- Decision box (diamond) is FLOWCHART, NOT part of DFD
- LOC is language-DEPENDENT; FP is language-INDEPENDENT
- CORRECTIVE maintenance = fixes bugs AFTER delivery (not during dev)
- Integration testing finds INTERFACE errors (not design/analysis errors)
- COCOMO by Barry Boehm (NOT Roger Pressman or Ian Sommerville)
- Context diagram = Level 0 DFD (NOT Level 1)
- COCOMO formula: $\text{Effort} = a \times (\text{KLOC})^b$ (NOT $a + \text{KLOC}$)
- CMM has 5 levels (NOT 4 or 6)

QUICK ANSWERS (2-MARK)

- COCOMO stands for? → CONstructive COst Model (Barry Boehm, 1981)
- Worst coupling? → Content coupling
- Best cohesion? → Functional cohesion
- Context diagram = ? → Level 0 DFD
- Main purpose of integration testing? → Find interface errors
- SRS also known as? → Black-box specification
- NOT a SRS characteristic? → Ambiguous
- CMM levels? → 5: Initial, Repeatable, Defined, Managed, Optimizing
- Spiral model dimensions? → Radial and Angular
- Most important Spiral feature? → Risk Management
- FP measures? → Software size from user/business perspective
- Regression testing done? → After bug fixes or code changes
- Musa model used for? → Software reliability estimation
- Preventive maintenance? → Changes to reduce future system failures

LAST MINUTE POINTS

- ★ Waterfall (7x repeated) = MUST KNOW all 6 phases perfectly
- ★ Black/White box (6x) = know EP, BVA for BB; Statement/Branch/Path for WB
- ★ SRS (6x) = IEEE: Introduction + Overall Desc + Specific Requirements
- ★ COCOMO formula: $\text{Effort} = a \times (\text{KLOC})^b$
- ★ Coupling best→worst: Data→Stamp→Control→Common→Content
- ★ Cohesion worst→best:
Coincidental→Logical→Temporal→Procedural→Communicational→Sequential→Functional
- ★ CMM 5 levels: Initial, Repeatable, Defined, Managed, Optimizing
- ★ 4 types maintenance: Corrective(21%), Adaptive(25%), Perfective(50%), Preventive(4%)
- ★ DFD 4 components: Process, Data Flow, Data Store, External Entity
- ★ Spiral 4 quadrants: Planning, Risk Analysis, Development, Review
- ★ LOC = language dependent | FP = language independent
- ★ Content coupling = WORST | Functional cohesion = BEST